

P3296R4 let_async_scope

Date: 17th March 2025

Author: Anthony Williams anthony@justsoftwaresolutions.co.uk

Audience: SG1, LEWG

Background and motivation

This is intended to address concerns raised in LEWG about ensuring that a `counting_scope` (see [P3149R3](#)) is joined: the scope provided by `let_async_scope` is always joined, irrespective of how the nested work completes, and whether or not the provided function throws an exception after spawning work.

Code with explicit `counting_scope`:

```
some_data_type scoped_data = make_scoped_data();
counting_scope scope;

spawn(on(exec, [&] {
    spawn(on(exec, [&] {
        if (need_more_work(scoped_data)) {
            spawn(on(exec, [&] { do_more_work(scoped_data); }), scope);
            spawn(on(exec, [&] { do_more_other_work(scoped_data); }), scope);
        }
    }), scope);
    spawn(on(exec, [&] { do_something_else_with(scoped_data); }), scope);
}), scope);

maybe_throw();

this_thread::sync_wait(scope.join());
```

Here, if `maybe_throw` throws an exception, then the scope is not joined, and the nested tasks can continue executing asynchronously, potentially accessing both the scope and `scoped_data` objects out of lifetime.

Using `let_async_scope` addresses this by encapsulating the scope object and the result of the previous sender. The returned sender does not complete until all tasks nested on the scope complete, even if the function passed to `let_async_scope` exits via an exception:

```
auto scope_sender = just(make_scoped_data()) | let_async_scope([](auto scope_token,
                                                                auto& scoped_data) {
    spawn(on(exec, [scope_token, &scoped_data] {
        spawn(on(exec, [scope_token, &scoped_data] {
            if (need_more_work(scoped_data)) {
                spawn(on(exec, [&scoped_data] { do_more_work(scoped_data); }), scope_token);
                spawn(on(exec, [&scoped_data] { do_more_other_work(scoped_data); }), scope_token);
            }
        }), scope_token);
        spawn(on(exec, [&scoped_data] { do_something_else_with(scoped_data); }), scope_token);
    }), scope_token);
    maybe_throw();
});

this_thread::sync_wait(scope_sender);
```

Here, even if `maybe_throw` throws an exception, then `scope_sender` doesn't complete until all the nested tasks have completed. This prevents out-of-lifetime access to the `scoped_data` or the scope itself, unless references to the data or `scope_token` are stored outside the sender tree.

Stop requests are propagated to all senders nested in the async scope, but does not prevent those senders adding additional work to the scope. This allows senders to respond to stop requests by scheduling additional work to perform the necessary cleanup for cancellation.

If either the function passed to `let_async_scope` throws an exception, or any of the senders associated with the async scope complete with an error, then that exception or error completion is used as the completion of the sender returned from `let_async_scope`. This applies even if the function passed to `let_async_scope` returns normally; in that case the return value is discarded in favour of the

error return. If multiple errors are raised, one of them is chosen to be used as the completion of the sender returned from `let_async_scope`; all other errors are discarded.

Given:

```
auto scope_sender = just(make_scoped_data()) | let_async_scope([](auto scope_token,
                                                                    auto& scoped_data) {
    spawn(just_error(foo{}), scope_token);
    spawn(just_error(bar{}), scope_token);
});

this_thread::sync_wait(scope_sender);
```

Then the `sync_wait` will throw either an exception of type `foo` or an exception of type `bar`, but it is not specified which.

In order to allow the error propagation, all senders associated with the scope must have a compatible error signature. The default error signature is `set_error_t(std::exception_ptr)`, and all raised errors are wrapped with AS-EXCEPT-PTR (See [exec.general/8]). An explicit error signature can be specified by calling `let_async_scope_with_error`, in which case errors are *not* converted unless `std::exception_ptr` is the only permitted error type.

“Noexcept” error signature scopes:

```
auto scope_sender = just(make_scoped_data()) | let_async_scope_with_error<>([](auto scope_token,
                                                                    auto& scoped_data) noexcept {
    spawn(just_error(foo{}), scope_token); // error, sender may fail in "noexcept" scope
});

this_thread::sync_wait(scope_sender);
```

“Default” error signature scopes:

```
auto scope_sender = just(make_scoped_data()) | let_async_scope([](auto scope_token,
                                                                    auto& scoped_data) noexcept(false) {
    spawn(just_error(foo{}), scope_token); // Coerced with AS-EXCEPT-PTR
});

this_thread::sync_wait(scope_sender); // throws foo{}
```

“Explicit error signature” scopes:

```
auto scope_sender = just(make_scoped_data()) |
    let_async_scope_with_error<foo,bar>(
    [](auto scope_token, auto& scoped_data) noexcept {
        spawn(just_error(foo{}), scope_token); // OK
        spawn(just_error(bar{}), scope_token); // OK
        spawn(just_error(baz{}), scope_token); // error
    });

this_thread::sync_wait(scope_sender | upon_error([](auto e){
    static_assert(is_same<decltype(e), foo> || is_same<decltype(e), bar>,
        "Error must be foo or bar");
}));
```

Proposal

`let_async_scope` provides a means of creating an async scope (see [P3149R6](#)), which is associated with a set of tasks, and ensuring that they are all complete before the async scope sender completes. The previous sender’s result is passed to a user-specified invocable, along with an async scope token, which returns a new sender that is connected and started.

The sender returned by `let_async_scope` completes with the result of the completion of the sender returned from the supplied invocable. It does not complete until all tasks nested on the `scope_token` passed to the invocable have completed. Additional tasks may be nested on copies of the `scope_token`, even if the initial sender returned from the invocable has completed. The returned `scope_sender` will not complete while there are any nested tasks that have not completed.

If the callable supplied to `let_async_scope` does not return a sender, it must return `void`. The sender returned from `let_async_scope` will then have a `void` value completion.

Stop requests are propagated to all senders nested in the async scope.

The environment from the receiver connected to the sender returned from `let_async_scope` is propagated via the internal receiver to all senders spawned using the supplied scope token.

If the callable passed to `let_async_scope` throws an exception, or any of the senders associated with the scope complete with an error, then a stop request is propagated to all outstanding senders spawned using the scope token.

If either the function passed to `let_async_scope` throws an exception, or any of the senders associated with the async scope complete with an error, then that exception or error completion is used as the completion of the sender returned from `let_async_scope`. This applies even if the function passed to `let_async_scope` returns normally; in that case the return value is discarded in favour of the error return. If multiple errors are raised, one of them is chosen to be used as the completion of the sender returned from `let_async_scope`; all other errors are discarded.

If the error list specified for `let_async_scope_with_error` does not specify `std::exception_ptr`, then the function supplied must be declared `noexcept`, otherwise the program is ill-formed.

If the possible error completions of senders passed to `spawn` with a token from `let_async_scope_with_error` are not compatible with the error signatures then the program is ill-formed.

Wording

execution::let_async_scope

1. `let_async_scope` transforms a sender's value completions into a new child asynchronous operation associated with an async scope, by passing the sender's result datums to a user-specified callable, which returns a new sender that is connected and started.
2. For a subexpression `sndr`, let `let-async-scope-env(sndr)` be expression-equivalent to the first well-formed expression below:

- `SCHED-ENV(get_completion_scheduler<decayed-typeof<set_value>>(get_env(sndr)))`
- `MAKE-ENV(get_domain, get_domain(get_env(sndr)))`
- `env<>{}`

3. The expression `let_async_scope(sndr, f)` is expression-equivalent to:

```
let_async_scope_with_error<std::exception_ptr>(sndr, f);
```

4. The expression `let_async_scope_with_error<Errors...>(sndr, f)` is expression-equivalent to:

```
transform_sender(
    get-domain-early(sndr),
    make-sender(let_async_scope_with_error<Errors...>, f, sndr));
```

Where `Errors` is the list of possible error completion types.

5. The exposition-only class template `impls-for` (`[exec.snd.general]`) is specialized for `let_async_scope` as follows:

```
namespace std::execution {
    template<class Errors, class State, class Rcvr, class... Args>
    void let-async-scope-bind(State& state, Rcvr& rcvr, Args&&... args); // exposition only

    template<typename ... Errors>
    struct impls-for<decayed-typeof<let_async_scope_with_error<Errors...>>> : default-impls {
        static constexpr auto get-state = see below;
        static constexpr auto complete = see below;
    };
}
```

1. Let `receiver2` denote the following exposition-only class template:

```
namespace std::execution {
    template<class Rcvr, class Env>
    struct receiver2 : Rcvr {
        explicit receiver2(Rcvr rcvr, Env env)
            : Rcvr(std::move(rcvr)), env(std::move(env)) {}
    };
}
```

```

    auto get_env() const noexcept {
        const Rcvr& rcvr = *this;
        return JOIN-ENV(env, FWD-ENV(execution::get_env(rcvr)));
    }

    Env env; // exposition only
};
}

```

2. `impls-for<decayed-typeof<let_async_scope_with_error<Errors...>>::get-state` is initialized with a callable object equivalent to the following:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) requires see below {
    auto&& [tag, data, child] = std::forward<Sndr>(sndr);
    return [&]<class Fn, class Env>(Fn fn, Env env) {
        using args-variant-type = see below;
        using ops2-variant-type = see below;
        using scope-type = see below;

        struct state-type { // exposition only
            Fn fn;
            Env env;
            scope-type scope;
            args-variant-type args;
            ops2-variant-type ops2;
            optional<error-variant-type> error;
        };
        return state-type{std::move(fn), std::move(env), {}, {}};
    }(std::forward_like<Sndr>(data), let_async_scope_env(child));
}

```

1. `scope-type` is a type that behaves like a `counting_scope`, except that it is never *closed*.
2. `scope-token-type` is the type of the `async-scope-token` associated with this invocation of `let_async_scope_with_error`. It is a type that satisfies the `async_scope_token` concept, and wraps an instance of the `scope-type::token` associated with the internal `scope-type`, such that calls to the `wrap`, `try_associate` and `disassociate` member functions are forwarded to the wrapped `scope-type::token` instance.
3. Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, decltype(FWD-ENV(get_env(dec_lval<Rcvr>()))>>`. Let `LetSigs` be a pack of those types in `Sigs` with a return type of `decayed-typeof<set_value>`. Let `as-tuple` be an alias template such that `as-tuple<Tag(Args...)>` denotes the type `decayed-tuple<Args...>`. Then `args-variant-type` denotes the type `variant<monostate, as-tuple<LetSigs>...>` with duplicate types removed.
4. Let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<Fn, scope-token-type, decay_t<Args>&...>`. Then `ops2-variant-type` denotes the type `variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, Env>>...>` with duplicate types removed.
5. The *requires-clause* constraining the above lambda is satisfied if and only if the types `args-variant-type` and `ops2-variant-type` are well-formed.
6. `error-variant-type` is a `variant<Errors...>`, with duplicate types removed.
7. `scope-token-type` shall be a unique type, such that invoking `spawn(snd, token, env)` or `spawn(snd, token)` where `token` is an instance of `scope-token-type` invokes a distinct overload of `spawn`. Such an invocation is ill-formed if the completion signatures of `snd` include error completions that are not compatible with the `Errors...` list of the `let_async_scope_with_error<Errors...>` invocation. If the error list is compatible, then such an invocation of `spawn(snd, token, env)` is equivalent to

```

spawn(snd | upon_error(
    [&state](auto&& error){
        {
            internal-synchronization-lock guard(state);
            state.errors.emplace(TRANSFORM-ERROR(error));
        }
        state.scope.request_stop();
    }, state.scope.get_token(), env);

```

and an invocation of `spawn(snd, token)` is equivalent to

```

spawn(snd | upon_error(
    [&state](auto&& error){
        {
            internal-synchronization-lock guard(state);
            state.errors.emplace(TRANSFORM-ERROR(error));
        }
        state.scope.request_stop();
    }, state.scope.get_token(), state.env);

```

Where TRANSFORM-ERROR is AS-EXCEPT-PTR(error) if Errors... is a list consisting of the single element std::exception_ptr, and std::forward<decltype(error)>(error) otherwise,

and internal-synchronization-lock guard(state); is an exposition only placeholder for appropriate synchronization to ensure that the emplacement to state.errors is correctly synchronized and does not race.

[Note: This could be achieved by storing a mutex in the state and using lock_guard, but other implementation strategies may be more optimal. — end note]

3. The exposition-only function template let-async-scope-bind is equal to:

```

auto& args = state.args.emplace<decayed-tuple<scope-token-type, Args...>>(
    create-scope-token(), std::forward<Args>(args)...);
try {
    auto sndr2 = nest(apply(std::move(state.fn), args), state.scope);
    auto join_sender = state.scope.join();
    auto result_sender = when_all_with_variant(std::move(sndr2), std::move(join_sender)) |
        then([&](auto& result, auto&) { return result; });
    auto rcvr2 = receiver2{std::move(rcvr), std::move(state.env)};
    auto mkop2 = [&] { return connect(std::move(result_sender), std::move(rcvr2)); };
    auto& op2 = state.ops2.emplace<decltype(mkop2)>(emplace-from{mkop2});
    start(op2);
} catch (...) {
    state.scope.request_stop();
    auto result_sender = when_all(just_error(std::current_exception()), state.scope.join());
    auto rcvr2 = receiver2{std::move(rcvr), std::move(state.env)};
    auto mkop2 = [&] { return connect(std::move(result_sender), std::move(rcvr2)); };
    auto& op2 = state.ops2.emplace<decltype(mkop2)>(emplace-from{mkop2});
    start(op2);
}

```

where create-scope-token() creates an instance of the scope-token-type associated with the state for this invocation of let_async_scope_with_error.

4. impls-for<decayed-typeof<let_async_scope_with_error<Errors...>>::complete is initialized with a callable object equivalent to the following:

```

[]<class Tag, class... Args>
(auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept -> void {
    if constexpr (same_as<Tag, decayed-typeof<set_value>>) {
        TRY-EVAL(std::move(rcvr), let-async-scope-bind(state, rcvr, std::forward<Args>(args)...));
    } else {
        Tag()(std::move(rcvr), std::forward<Args>(args)...);
    }
}

```

6. Let sndr and env be subexpressions, and let Sndr be decltype((sndr)). If sender-for<Sndr, decayed-typeof<let_async_scope_with_error<Errors...>> is false, then the expression let_async_scope_with_error<Errors...>.transform_env(sndr, env) is ill-formed. Otherwise, it is equal to JOIN-ENV(let-env(sndr), FWD-ENV(env)).

7. Let the subexpression out_sndr denote the result of the invocation let_async_scope_with_error<Errors...>(sndr, f) or an object copied or moved from such, and let the subexpression rcvr denote a receiver such that the expression connect(out_sndr, rcvr) is well-formed. The expression connect(out_sndr, rcvr) has undefined behavior unless it creates an asynchronous operation ([async.ops]) that, when started:

- invokes f when set_value is called with sndr's result datums,
- makes its completion dependent on the completion of a sender returned by f, and
- propagates the other completion operations sent by sndr.

Revision History

R1

- Improved examples, added links

R2

- Propagate environment to spawned senders associated with the scope.
- Errors lead to stop requests being sent to all senders associated with the scope.
- An Error is stored in the scope and the scope sender completes with an error if there is one
- Allowed errors can be specified

R3

- Fix specification of allowed errors
- Make it clear that a new overload of `spawn` is expected.

R4

- Fix specification of the `async` scope type
- Two overloads of `spawn`: with/without explicit environment
- Remove explicit mutex from the state

Acknowledgements

Thanks to Ian Petersen, Lewis Baker, Inbal Levi, Kirk Shoop, Eric Niebler, Ruslan Arutyunyan, Maikel Nadolski, Lucian Radu Teodorescu, Robert Leahy, Dmitry Prokoptsev, and everyone else who contributed to discussions leading to this paper, and commented on early drafts.